



eSim Summer Fellowship 2026

Submission Task 5

Project Report

On

Automated Tool Manager for eSim

Submitted by

MAUSAM KAR

Department of Computer Science and Engineering
VIT Bhopal University

April 2026

Declaration

I hereby declare that the project report titled **"Automated Tool Manager for eSim"**, submitted by me for the eSim Summer Fellowship 2026, is a record of genuine work carried out by me as part of my academic and technical work at the Department of Computer Science and Engineering, VIT Bhopal University.

The work presented in this report has not been previously submitted for any other purpose. All sources, both primary and secondary, have been duly acknowledged.

Name: MAUSAM KAR
Date: April 2026
Signature: _____

Acknowledgement

I would like to express my profound gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I am deeply indebted to **Prof. Prabhu Ramachandran** and the entire team at **FOSSEE (Free and Open Source Software in Education)** for providing me with the opportunity to work on this meaningful project as part of the eSim Summer Fellowship 2026. Their continuous guidance and support have been invaluable throughout this journey.

I am also thankful to the Department of Computer Science and Engineering at **VIT Bhopal University** for maintaining an environment conducive to learning and innovation.

My sincere thanks go to the eSim development team for creating such a powerful open-source EDA tool, and to the open-source community whose contributions make projects like this possible.

Finally, I would like to acknowledge my institution, **Vellore Institute of Technology**, for encouraging research and development activities among its students.

MAUSAM KAR
April 2026

Abstract

The Automated Tool Manager for eSim is a comprehensive solution designed to address the challenges associated with managing external tools and dependencies in the eSim EDA platform. This project implements a modular, cross-platform tool management system that automates the installation, configuration, update, and removal of external tools required by eSim for circuit design, simulation, and PCB design workflows.

The tool manager provides both a Command-Line Interface (CLI) built using Python's Typer framework and a Graphical User Interface (GUI) developed with CustomTkinter, ensuring accessibility for users with varying technical backgrounds. It integrates seamlessly with popular package managers including apt (Debian/Ubuntu), Homebrew (macOS), and Chocolatey (Windows), while also supporting direct URL-based installations as a fallback mechanism.

Key features implemented include automated dependency checking and resolution, version management to ensure compatibility between tools, environment variable configuration for seamless tool integration, comprehensive logging for audit trails, and a verification system to confirm successful installations. The modular architecture enables easy extension and maintenance, supporting the addition of new tools without modifying the core system.

This report documents the complete design, implementation, and testing of the Automated Tool Manager, demonstrating its effectiveness in simplifying tool management for eSim users across Linux, Windows, and macOS platforms.

Keywords: eSim, EDA Tool Manager, Python, Cross-Platform, Package Management, Dependency Resolution, Typer, CustomTkinter

Contents

Declaration i

Acknowledgement ii

Abstract iii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Project Objectives . . . 2
- 1.4 Project Scope . . . 2
- 1.5 Report Structure . . . 2

2 Literature Survey 3
- 2.1 Overview of Electronic Design Automation Tools . . . 3
  - 2.1.1 eSim Platform . . . 3
  - 2.1.2 Related Open-Source EDA Tools . . . 3
- 2.2 Existing Package Management Solutions . . . 4
  - 2.2.1 Linux Package Managers . . . 4
  - 2.2.2 MacOS Package Managers . . . 4
  - 2.2.3 Windows Package Managers . . . 4
- 2.3 Technology Stack Analysis . . . 5
  - 2.3.1 Python as the Implementation Language . . . 5
  - 2.3.2 CLI Framework: Typer vs Alternatives . . . 5
  - 2.3.3 GUI Framework: CustomTkinter vs Alternatives . . . 5
- 2.4 Gaps in Existing Solutions . . . 6

3 System Architecture 7
- 3.1 Architectural Overview . . . 7
- 3.2 Architecture Diagram . . . 8
- 3.3 Module Descriptions . . . 8
  - 3.3.1 CLI Module (cli.py) . . . 8
  - 3.3.2 GUI Module (gui.py) . . . 9
  - 3.3.3 Installer Module (installer.py) . . . 9
  - 3.3.4 Updater Module (updater.py) . . . 9
  - 3.3.5 Dependency Checker Module (dependency_checker.py) . . . 9
  - 3.3.6 Environment Manager Module (env_manager.py) . . . 10
- 3.4 Data Flow Architecture . . . 10

3.5 Package Bridge Module (package_bridge.py) . . . 10
3.6 Configuration Module (config.py) . . . 11
3.7 Logger Module (logger.py) . . . 11
3.8 Data Storage Architecture . . . 11
3.8.1 Tools Registry (tools_registry.json) . . . 11
3.8.2 User Configuration (config.json) . . . 11

4 Requirement Analysis **12**
4.1 Functional Requirements . . . 12
4.1.1 Tool Installation Management . . . 12
4.1.2 Version Control . . . 12
4.1.3 Update and Upgrade System . . . 12
4.1.4 Configuration Handling . . . 13
4.1.5 Dependency Management . . . 13
4.1.6 User Interface . . . 13
4.2 Non-Functional Requirements . . . 13
4.2.1 Performance . . . 13
4.2.2 Reliability . . . 13
4.2.3 Usability . . . 13
4.2.4 Compatibility . . . 14
4.2.5 Maintainability . . . 14
4.3 User Stories . . . 14
4.3.1 New User Scenario . . . 14
4.3.2 Power User Scenario . . . 14
4.3.3 Troubleshooting Scenario . . . 14
4.4 Use Case Diagram . . . 15

5 Design and Implementation **16**
5.1 Design Principles . . . 16
5.1.1 Modularity . . . 16
5.1.2 Layered Architecture . . . 16
5.1.3 Cross-Platform Compatibility . . . 16
5.2 Implementation Details . . . 16
5.2.1 Installation Engine . . . 16
5.2.2 Version Comparison . . . 17
5.2.3 Dependency Resolution . . . 17
5.2.4 Environment Configuration . . . 18
5.3 Database Schema . . . 18
5.3.1 Tools Registry Schema . . . 18
5.3.2 User Configuration Schema . . . 18
5.4 Error Handling Strategy . . . 19
5.4.1 Subprocess Failures . . . 19
5.4.2 Network Errors . . . 19
5.4.3 Configuration Errors . . . 19

6 User Interface Design **20**
6.1 Design Philosophy . . . 20
6.2 Command-Line Interface Design . . . 20
6.2.1 CLI Output Formats . . . 20
6.2.2 CLI Command Structure . . . 21
6.2.3 CLI Operation Examples . . . 21
6.3 Graphical User Interface Design . . . 22
6.3.1 GUI Application Window . . . 22
6.3.2 GUI Features . . . 23
6.3.3 GUI Color Scheme . . . 23
6.4 Verification & Logging Interface . . . 23

7 Testing and Results **24**
7.1 Testing Strategy . . . 24
7.1.1 Unit Testing . . . 24
7.1.2 Integration Testing . . . 24
7.2 Test Results . . . 24
7.3 Platform Compatibility Testing . . . 25
7.4 Performance Testing . . . 25

8 Installation and Usage Guide **26**
8.1 Installation . . . 26
8.1.1 Prerequisites . . . 26
8.1.2 Quick Start . . . 26
8.2 Basic Usage . . . 27
8.2.1 Listing Available Tools . . . 27
8.2.2 Installing a Tool . . . 27
8.2.3 Installing All Tools . . . 27
8.2.4 Checking for Updates . . . 27
8.2.5 Updating Tools . . . 27
8.2.6 Checking Dependencies . . . 28
8.2.7 Verifying Installation . . . 28
8.2.8 Viewing Logs . . . 28
8.3 GUI Usage . . . 28
8.4 Configuration . . . 28
8.4.1 Viewing Configuration . . . 28
8.4.2 Setting Configuration Values . . . 29
8.5 Troubleshooting . . . 29
8.5.1 Common Issues . . . 29

9 Scopes for Improvement **30**
9.1 Project Summary . . . 30
9.1.1 Achievements . . . 30
9.1.2 Learnings . . . 30
9.2 Scopes for Improvement . . . 30
9.2.1 Docker Support for Improved Isolation . . . 31
9.2.2 Extensible Plugin Architecture . . . 31
9.2.3 Proactive Auto-Repair Capabilities . . . 31

9.2.4 Remote Web Interface and Analytics 31

10 Docker Containerization **32**
10.1 Overview and Motivation 32
10.2 Why Docker for This Project? 32
10.3 Dockerfile Architecture 33
10.4 Build and Usage 33
10.4.1 Building the Docker Image 33
10.4.2 Running the Container 34
10.5 Docker Build Snapshots 34
10.6 Advantages Over Native Installation 35

11 Conclusion **36**

12 References **37**

List of Figures

2.1 Open-Source EDA Ecosystem and Integration Context 4
2.2 Technology Selection Rationale and Trade-offs 6

3.1 System Architecture of Automated Tool Manager 8
3.2 Workflow and Control Flow of Tool Manager 10

4.1 Use Case Diagram for Automated Tool Manager 15

6.1 CLI Command Hierarchy 21
6.2 CLI Options and Commands Help Menu 21
6.3 CLI Tool Listing and Status Table 22
6.4 GUI Main Window Layout and Operation Dashboard 22
6.5 Dependency Check and Verification Output 23

10.1 Dockerfile Generation — Auto-generated Dockerfile via `esim-tool generate-docker` Command 34
10.2 Docker Build — Image Build Process Showing Layer Caching and Package Installation 35

List of Tables

7.1 Unit Test Results Summary 25
7.2 Platform Compatibility Matrix 25

10.1 Native Installation vs Docker Container Deployment 35

Chapter 1

Introduction

1.1 Background

In the field of electronic design automation (EDA), eSim has emerged as a powerful open-source tool that provides capabilities for circuit design, simulation, analysis, and PCB design. It serves engineers, researchers, educators, and hobbyists who need an accessible platform for electronic circuit development without the overhead of commercial EDA software licensing costs.

The eSim platform relies on several external tools and libraries to provide its full range of functionality. These include circuit simulators like Ngspice for SPICE simulation, PCB design tools like KiCad for board layout, and Modelica-based simulation environments like OpenModelica for advanced modeling. Each of these tools has its own installation procedures, dependency requirements, version constraints, and configuration needs.

Managing these external tools manually presents significant challenges. Installation procedures are often complex and require system-level permissions, dependency chains can break silently when versions drift, and compatibility between eSim and external tools must be preserved over time. In addition, installation workflows vary considerably across Linux, Windows, and macOS, which makes cross-platform support difficult for both new and experienced users. Regular updates add another layer of operational overhead because users must upgrade tools without breaking existing project environments.

1.2 Problem Statement

The manual management of external tools for eSim creates a barrier to entry for new users and ongoing frustration for existing users. A user attempting to set up eSim for the first time must:

1. Identify all required external tools and their compatible versions.
2. Locate and download the appropriate packages for their operating system.
3. Resolve complex dependency relationships between libraries.
4. Configure environment variables and system paths correctly.
5. Verify that all tools are accessible and working properly.

6. Manually check for and apply updates while ensuring compatibility.

This complexity detracts from the core value proposition of eSim - enabling users to focus on circuit design rather than tool management. There is a clear need for an automated solution that handles these operational complexities on behalf of the user.

1.3 Project Objectives

The primary objective of this project is to design and implement an **Automated Tool Manager** for eSim that makes tool setup reliable and low-friction. The system automates installation with minimal user intervention, validates compatibility before changes are applied, manages updates safely, configures required environment variables and paths, and resolves dependencies in a predictable way. It also offers both CLI and GUI interaction models so that the same core functionality is accessible to users with different workflows, while maintaining consistent behavior across Linux, Windows, and macOS.

1.4 Project Scope

The scope of this project includes the complete design and implementation of a modular tool management architecture, installation workflows based on package managers and direct downloads, automated dependency checking, and environment configuration support. It also includes both CLI and GUI interfaces, cross-platform validation, and full technical documentation covering architecture, usage, and implementation behavior.

1.5 Report Structure

This report is organized as a progressive narrative. Chapter 1 introduces the problem context and project goals, while Chapter 2 surveys related tools and technologies that informed implementation decisions. Chapter 3 presents architecture, Chapter 4 captures requirements, and Chapter 5 explains implementation details. Chapter 6 documents interface design for both CLI and GUI workflows, Chapter 7 presents testing and results, Chapter 8 serves as a practical installation and usage guide, and Chapter 9 concludes with key outcomes and future extensions.

Chapter 2

Literature Survey

2.1 Overview of Electronic Design Automation Tools

Electronic Design Automation (EDA) tools have revolutionized the way electronic circuits are designed, simulated, and manufactured. The evolution from manual schematic drawing to computer-aided design has dramatically increased productivity and enabled the design of increasingly complex integrated circuits and electronic systems.

Open-source EDA tools have played a crucial role in democratizing access to electronic design capabilities. Projects like eSim, KiCad, and OpenROAD have demonstrated that high-quality design tools can be developed through collaborative open-source development.

2.1.1 eSim Platform

eSim is a Qt-based EDA tool with an integrated environment for schematic capture, simulation, and PCB design. It provides a schematic editor for circuit entry, simulator integration through Ngspice, PCB layout support for board-level design, and model development capabilities for advanced simulation workflows.

eSim's architecture leverages several external tools to provide these capabilities, making tool management a critical operational concern.

2.1.2 Related Open-Source EDA Tools

The broader open-source EDA ecosystem includes KiCad for schematic and PCB workflows, Qucs for circuit simulation with a GUI-oriented experience, OpenROAD for digital design automation, and Ngspice as a widely used mixed-level SPICE simulator. These tools demonstrate the maturity of open tooling while also highlighting the integration complexity that this project addresses.

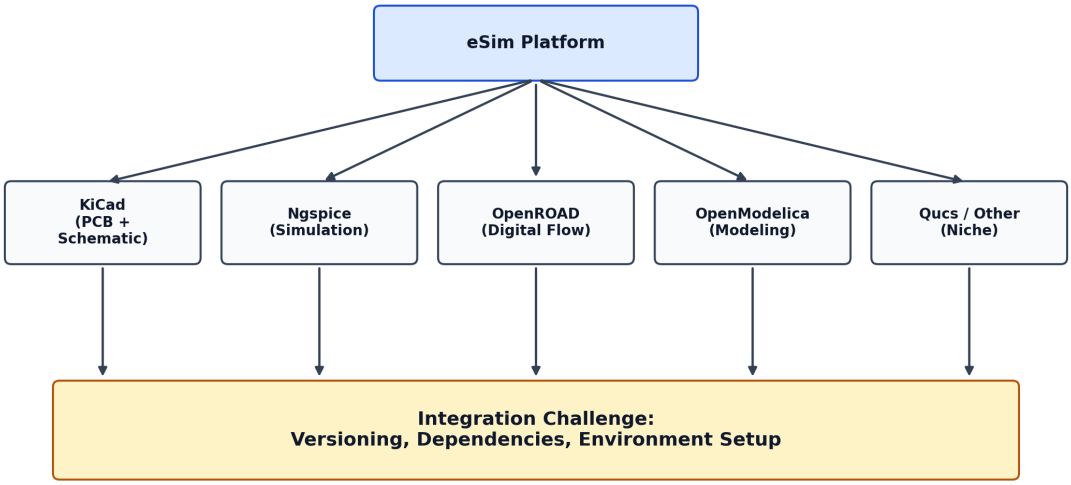


Figure 2.1: Open-Source EDA Ecosystem and Integration Context

2.2 Existing Package Management Solutions

The concept of automated package management is well-established in software development. Understanding existing solutions provides insights applicable to the tool manager design.

2.2.1 Linux Package Managers

APT (Advanced Package Tool) is the primary package management system for Debian-based distributions and provides robust dependency resolution and repository-based installation. Red Hat based ecosystems rely on YUM or DNF with similar objectives but different package formats and repository conventions, while Arch Linux uses Pacman with a minimal and performance-oriented design.

2.2.2 MacOS Package Managers

Homebrew is the dominant package manager on macOS and uses formula definitions to standardize install procedures. MacPorts offers an alternative approach that emphasizes source-based builds and a separate package ecosystem.

2.2.3 Windows Package Managers

On Windows, Chocolatey is widely used for scripted software installation and lifecycle management, winget provides Microsoft-native package operations, and Scoop offers a lightweight workflow that is popular among developers. Supporting this diversity is important for reliable cross-platform automation.

2.3 Technology Stack Analysis

The selection of an appropriate technology stack is crucial for project success. This section analyzes the technologies considered for the Automated Tool Manager.

2.3.1 Python as the Implementation Language

Python was selected as the primary implementation language because it offers a strong standard library for subprocess control, filesystem operations, and networking while retaining cross-platform consistency across Linux, Windows, and macOS. Its ecosystem also supports both CLI and GUI development in a single language, which helped keep the implementation cohesive. From a project delivery perspective, Python enabled faster iteration, easier maintenance, and readable modular code suitable for long-term extension.

2.3.2 CLI Framework: Typer vs Alternatives

For the command-line layer, Typer was preferred over Click and argparse because it provides a cleaner typed interface, better command discoverability, and auto-generated help that is useful for first-time users. Click remains a mature and flexible alternative, while argparse is lightweight and built-in, but Typer offered the best balance between developer productivity and professional CLI behavior for this project.

2.3.3 GUI Framework: CustomTkinter vs Alternatives

For the GUI, CustomTkinter was selected to achieve a modern look with minimal platform-specific complexity. Compared to PyQt/PySide, it reduces setup overhead and avoids additional packaging burden for this scope. Compared to wxPython, it offers cleaner visual theming out of the box. This made it suitable for a utility-style application where responsiveness, clarity, and consistency were more important than heavyweight widget ecosystems.

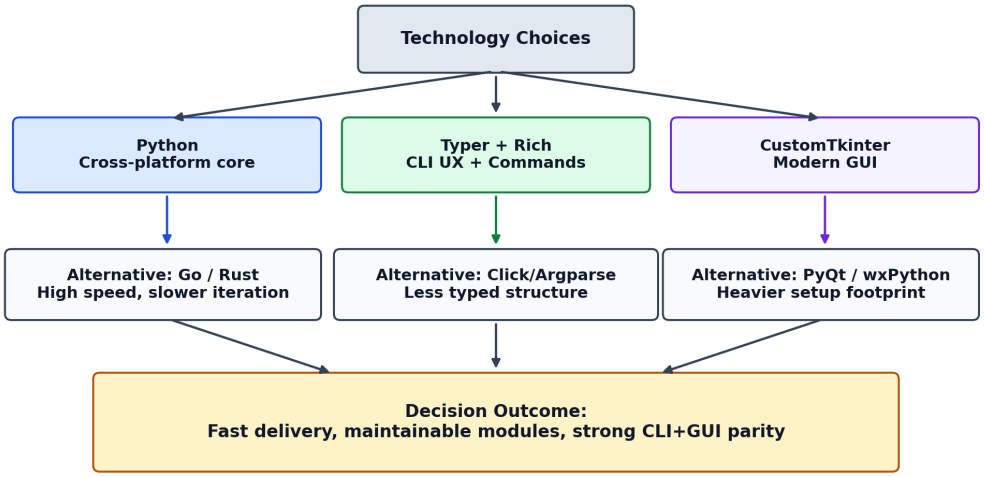


Figure 2.2: Technology Selection Rationale and Trade-offs

2.4 Gaps in Existing Solutions

Analysis of existing tools reveals four practical gaps that this work addresses directly. First, general-purpose package managers are not aware of eSim-specific tool compatibility and version constraints. Second, existing installers do not provide an integrated workflow aligned with eSim configuration needs. Third, user-facing diagnostics are often fragmented, which makes troubleshooting difficult for new users. Finally, most solutions do not include end-to-end verification that confirms tools are correctly installed and operational in the target environment.

Chapter 3

System Architecture

3.1 Architectural Overview

The Automated Tool Manager employs a modular, layered architecture that separates concerns and promotes maintainability. The architecture consists of four primary layers:

1. **User Interface Layer**: CLI and GUI components that handle user interaction.
2. **Business Logic Layer**: Core management engines for installation, updates, and configuration.
3. **Service Layer**: Supporting services for dependency checking, environment management, and logging.
4. **Data Layer**: Registry and configuration storage systems.

3.2 Architecture Diagram

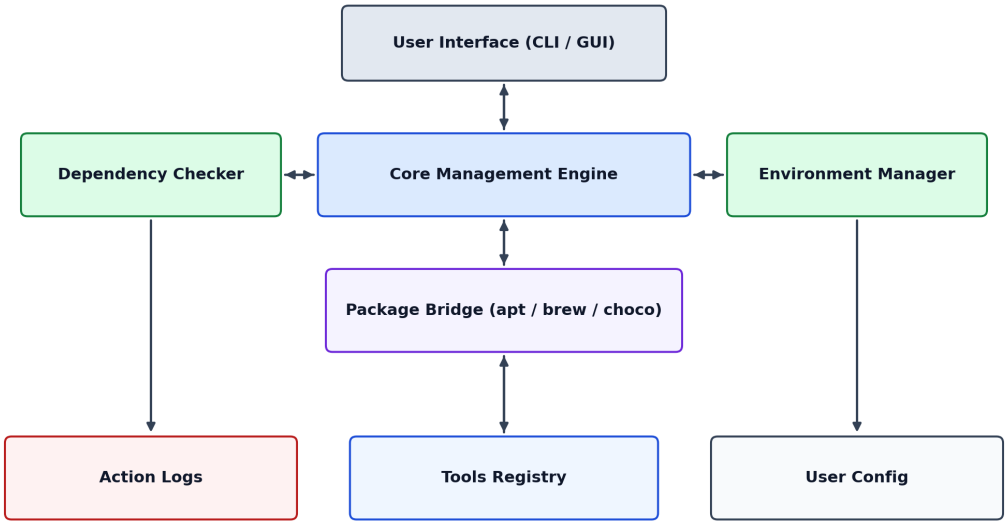


Figure 3.1: System Architecture of Automated Tool Manager

3.3 Module Descriptions

The system is composed of several interconnected modules, each responsible for specific functionality:

3.3.1 CLI Module (cli.py)

The Command-Line Interface module provides the primary interface for power users. It parses and validates user commands using the Typer framework, elegantly routing them to appropriate business logic functions. To ensure operations are clear, it formats and displays results using the Rich library for enhanced readability, provides progress indicators for long-running operations, and handles unexpected errors by presenting user-friendly messages.

Key commands include:

- `install` / `install-all`: For acquiring specified tools or installing all registered dependencies.
- `update` / `list`: For managing tool versions and viewing current operation status.
- `check-deps`, `verify`, and `logs`: For system diagnostics, validation, and comprehensive auditing.

3.3.2 GUI Module (gui.py)

The Graphical User Interface module provides an intuitive point-and-click experience built using CustomTkinter for a modern, native-looking appearance. It presents tools in a sortable table with clear status indicators, alongside real-time console output for operation feedback. The interface consistently supports dark and light themes for user preference, and includes progress bars to visually represent installation operations.

The GUI features a structured layout comprising a sidebar for quick access to common operations, a comprehensive tools table, a console panel for logs, and a dedicated progress indicator.

3.3.3 Installer Module (installer.py)

The core installation engine handles all aspects of acquiring and configuring external tools. First, it determines the correct version to install based on registry definition and user input, ensuring system dependencies are thoroughly validated before proceeding. It seamlessly integrates with appropriate package managers or falls back to direct URL downloads when native package managers are unavailable. Finally, it cleanly manages archive extraction, directory organization, and confirms success through automated verification checks.

The installer supports multiple installation methods:

1. **Primary**: Package manager installation (apt, brew, choco)
2. **Fallback**: Direct URL download and extraction
3. **Executable**: Silent installation for Windows .exe files

3.3.4 Updater Module (updater.py)

The update management module ensures all software stays current by continuously comparing installed versions against registry records. It dynamically scans all installed tools for available updates and can apply them with minimal user intervention. It is optimized to support batch operations to update all tools simultaneously if desired.

Key update operations involve checking individual or multiple tools through `check_tool_update(tool registry)` and seamlessly applying updates via `update_all_tools(registry)` when newer versions are identified.

3.3.5 Dependency Checker Module (dependency_checker.py)

The dependency management system proactively ensures all system requirements are met before allowing tool installation. Through initial auto-detection of the host operating system and accessible package managers, the system retrieves installed package versions and validates their compatibility against predetermined minimum requirements. If issues arise, it immediately provides clear, actionable feedback regarding missing or incompatible dependencies.

Platform-specific validation relies on native system utilities including:

- **Linux**: `dpkg-query` for Debian-based systems
- **macOS**: `brew list` for Homebrew packages

- **Windows**: `choco list` for Chocolatey

3.3.6 Environment Manager Module (env_manager.py)

The environment configuration module guarantees seamless system integration by adding newly installed tool directories to the system PATH while configuring crucial tool-specific environment variables. This cross-platform abstraction dynamically adapts modifying actions to each OS and automatically verifies that environment configurations are successfully confirmed post-installation.

To accomplish this, it modifies `.bashrc` and `.zshrc` files with proper export statements on Linux/macOS, while securely updating `HKCU\Environment` registry keys on Windows platforms.

3.4 Data Flow Architecture

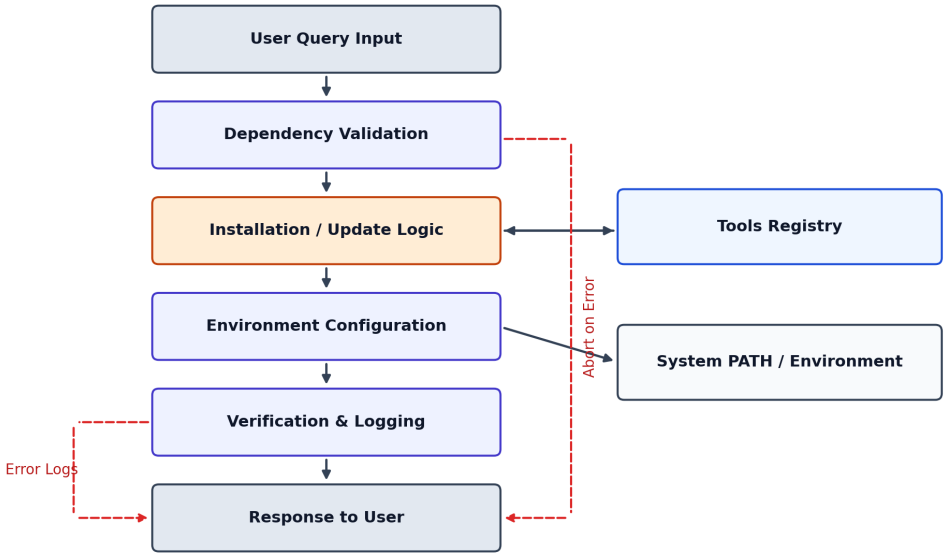


Figure 3.2: Workflow and Control Flow of Tool Manager

3.5 Package Bridge Module (package_bridge.py)

The Package Bridge establishes a unified, reliable interface for all system package managers. By intelligently auto-detecting the active package manager for the current OS, it provides consistent command syntax, predictably retrieves installed package versions in a standardized format, and elegantly abstracts platform-specific installation details away from the core execution engine.

Supported integrations include **apt** for Debian/Ubuntu environments, **brew** for macOS logic, and **choco** designed for Windows structures.

3.6 Configuration Module (config.py)

The configuration management system persistently stores and retrieves runtime user settings and application metadata. It flawlessly tracks user-configurable options alongside a robust installation history to precisely trace tool paths and package versions. Beyond user history, it provides sensible baseline default settings to guarantee a smooth and friction-less experience for new installations.

Active configurations are securely maintained in `/.esim_tool_manager/config.json`.

3.7 Logger Module (logger.py)

The sophisticated logging system preserves comprehensive audit trails by simultaneously writing exact timestamped entries to both standard console output and a dedicated log file. It fully leverages multiple systemic severity levels (DEBUG, INFO, WARNING, and ERROR) and accurately associates the specific source module for every entry to accelerate future troubleshooting and systemic clarity.

Log records are robustly written to `logs/esim_tool_manager.log`.

3.8 Data Storage Architecture

3.8.1 Tools Registry (tools_registry.json)

The central tools registry functions as the primary JSON database driving all interactions. It exhaustively defines available tools by capturing critical metadata such as proper display names, distinct categories, functional descriptions, recommended version information, and platform-specific installation instructions. Furthermore, it explicitly pairs vital underlying system dependencies, fallback download URLs, and specific environment variables directly with their corresponding tools.

3.8.2 User Configuration (config.json)

Locally-persisted user settings operate continuously to retain state. These include operational attributes such as the target eSim installation path (`esim_home`), preferences regarding automatic update checks, active installation methods, chosen logging verbosity levels, and a rich runtime dictionary of all successfully deployed tools.

Chapter 4

Requirement Analysis

4.1 Functional Requirements

The functional requirements for the Automated Tool Manager are derived from the core need to simplify tool management for eSim users.

4.1.1 Tool Installation Management

FR-001: The system shall provide automated download and installation of external tools.

FR-002: The system shall support installation via system package managers (apt, Homebrew, Chocolatey).

FR-003: The system shall provide fallback URL-based installation when package managers are unavailable.

FR-004: The system shall verify successful installation through automated checks.

FR-005: The system shall track installed versions and installation metadata.

4.1.2 Version Control

FR-006: The system shall store and manage version information for all registered tools.

FR-007: The system shall prevent installation of incompatible tool versions.

FR-008: The system shall support version resolution based on user input or registry defaults.

4.1.3 Update and Upgrade System

FR-009: The system shall check for available updates by comparing installed versions against the registry.

FR-010: The system shall provide functionality to update individual tools.

FR-011: The system shall provide functionality to update all installed tools simultaneously.

FR-012: The system shall notify users when updates are available.

4.1.4 Configuration Handling

FR-013: The system shall automatically configure PATH environment variables for installed tools.

FR-014: The system shall set tool-specific environment variables upon installation.

FR-015: The system shall persist configuration across sessions.

FR-016: The system shall adapt configuration methods to the host operating system.

4.1.5 Dependency Management

FR-017: The system shall check for required system dependencies before installation.

FR-018: The system shall verify dependency versions against minimum requirements.

FR-019: The system shall provide clear feedback about missing or incompatible dependencies.

FR-020: The system shall support forced installation despite dependency warnings.

4.1.6 User Interface

FR-021: The system shall provide a command-line interface for all core operations.

FR-022: The system shall provide a graphical interface for common operations.

FR-023: The system shall display installation progress during operations.

FR-024: The system shall provide formatted output for tool listings and status.

FR-025: The system shall maintain action logs for all operations.

4.2 Non-Functional Requirements

4.2.1 Performance

NFR-001: Tool listing shall complete within 2 seconds for registries with up to 100 tools.

NFR-002: Dependency checking shall complete within 5 seconds per tool.

NFR-003: The system shall support concurrent download operations where possible.

4.2.2 Reliability

NFR-004: The system shall not corrupt user configuration on unexpected termination.

NFR-005: The system shall provide clear error messages for all failure conditions.

NFR-006: The system shall maintain operation logs for troubleshooting.

4.2.3 Usability

NFR-007: The CLI shall provide comprehensive help documentation for all commands.

NFR-008: The GUI shall be intuitive and require no prior training to use.

NFR-009: Error messages shall be actionable and guide users toward resolution.

4.2.4 Compatibility

NFR-010: The system shall support Linux (Debian-based), Windows, and macOS.
NFR-011: The system shall adapt installation methods based on detected OS.
NFR-012: The system shall function with or without administrative privileges.

4.2.5 Maintainability

NFR-013: The system shall use modular architecture to facilitate updates.
NFR-014: The system shall separate user data from application code.
NFR-015: The system shall support addition of new tools through registry updates.

4.3 User Stories

4.3.1 New User Scenario

Story 1: As a new eSim user, I want to install all required tools with a single command so that I can start using eSim quickly.
Acceptance Criteria:

- User can run `esim-tool install-all`
- All dependencies are validated automatically
- User receives progress feedback during installation
- Installation completes without user intervention

4.3.2 Power User Scenario

Story 2: As an experienced user, I want to check for updates and apply them selectively so that I can maintain control over my tool versions.
Acceptance Criteria:

- User can run `esim-tool check-updates`
- Update information includes current and available versions
- User can update individual tools by name
- User can update all tools simultaneously

4.3.3 Troubleshooting Scenario

Story 3: As a user experiencing problems, I want to verify my installation and check logs so that I can diagnose issues.
Acceptance Criteria:

- User can run `esim-tool verify <tool>`
- User can run `esim-tool logs -tail N`

- Verification results indicate success or failure clearly
- Logs contain timestamps and detailed error information

4.4 Use Case Diagram

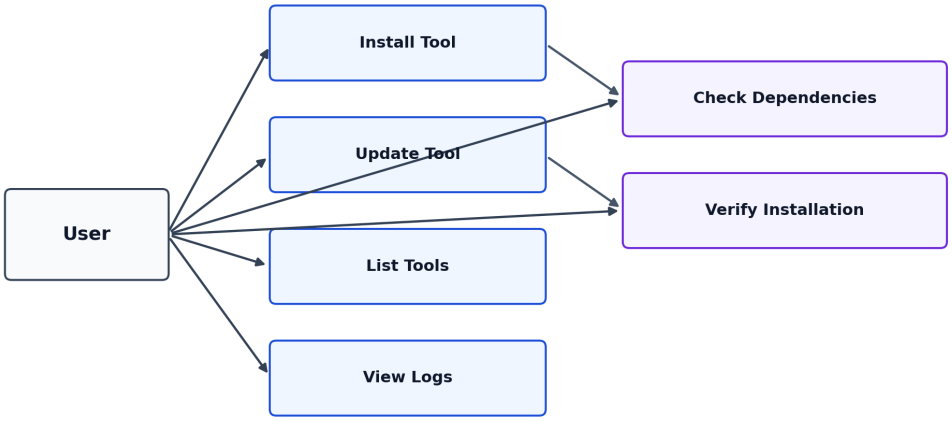


Figure 4.1: Use Case Diagram for Automated Tool Manager

Chapter 5

Design and Implementation

5.1 Design Principles

The design of the Automated Tool Manager follows several key principles:

5.1.1 Modularity

Each module has a single, well-defined responsibility so that behavior is easier to reason about and test in isolation. This separation improves debugging efficiency, supports long-term maintenance, enables reuse in multiple workflows, and also allows contributors to work in parallel without tight coupling.

5.1.2 Layered Architecture

The system is organized into clearly separated layers with explicit interfaces. The presentation layer handles user interaction through CLI and GUI surfaces, the business logic layer executes core operations, the service layer provides reusable system utilities, and the data layer manages persistent state and configuration.

5.1.3 Cross-Platform Compatibility

Design decisions prioritize portability by relying on Python abstractions where possible and confining platform-specific logic to dedicated modules. This approach keeps interfaces consistent across operating systems and reduces platform drift during development and testing.

5.2 Implementation Details

5.2.1 Installation Engine

The installation process follows a well-defined sequence:

1. **Input Validation**: Verify tool name exists in registry.
2. **Platform Check**: Confirm the tool supports the current OS.

3. **Dependency Validation**: Check system dependencies.
4. **Version Resolution**: Determine version to install.
5. **Existing Installation Check**: Detect if tool is already installed.
6. **Installation Method Selection**: Choose package manager or URL fallback.
7. **Execution**: Perform the installation.
8. **Verification**: Confirm installation succeeded.
9. **Configuration**: Apply environment settings.
10. **Recording**: Save installation metadata.

The implementation uses the Strategy pattern for installation methods: `install_via_package_manag` is used when native package managers are available, `install_via_url()` provides archive-based fallback installation, and `install_via_executable()` handles executable installers on Windows platforms.

5.2.2 Version Comparison

Version comparison uses the `packaging.version.parse()` function for robust semantic versioning:

```
from packaging.version import parse

def check_update_available(installed: str, latest: str) -> bool:
    try:
        return parse(latest) > parse(installed)
    except Exception:
        return False
```

Listing 5.1: Version Comparison Logic

This approach correctly handles multiple versioning styles, including standard semantic versions such as 1.0.0 and 2.1.3, Debian-style epochs such as 2:1.0.14, and pre-release variants such as alpha or beta tags.

5.2.3 Dependency Resolution

The dependency checker queries the system package manager to verify dependencies:

```
def _get_dep_installed_version(dep_name: str, system: str, pm: str) ->
   str | None:
    if system == "Linux":
        result = subprocess.run(
            ["dpkg-query", "-W", "-f=${Version}", dep_name],
            capture_output=True, text=True
        )
        if result.returncode == 0:
            return result.stdout.strip()
    # Similar for macOS (brew) and Windows (choco)
```

Listing 5.2: Dependency Version Query

5.2.4 Environment Configuration

Environment variables are set using OS-specific methods:

Linux/macOS:

```
# Appended to ~/.bashrc or ~/.zshrc
export NGSPICE_HOME=/usr/local/ngspice
export PATH="$PATH:$NGSPICE_HOME/bin"
```

Windows: Uses Windows Registry to set persistent environment variables at `HKCU\Environment`.

5.3 Database Schema

5.3.1 Tools Registry Schema

```
{
    "tools": [
        {
            "name": "ngspice",
            "display_name": "Ngspice",
            "category": "Simulator",
            "version": "41",
            "description": "Mixed-level/mixed-signal circuit simulator"
   ,
            "platforms": {
                "linux": {
                    "package_manager": "apt",
                    "package_name": "ngspice",
                    "fallback_url": "https://...",
                    "dependencies": ["libxaw7-dev"],
                    "env_vars": {"NGSPICE_HOME": "/usr"}
                },
                "windows": {...},
                "darwin": {...}
            }
        }
    ]
}
```

Listing 5.3: Tools Registry Structure

5.3.2 User Configuration Schema

```
{
    "esim_home": "~/eSim",
    "auto_update_check": true,
    "preferred_install_method": "package_manager",
    "log_level": "INFO",
    "installed_tools": {
        "ngspice": {
            "version": "41",
            "installed_at": "/usr/local/ngspice",
            "install_method": "apt",
            "platform": "linux"
```

```
        }
    }
}
```

Listing 5.4: User Configuration Structure

5.4 Error Handling Strategy

The system implements comprehensive error handling:

5.4.1 Subprocess Failures

Subprocess failures are handled by capturing non-zero exit codes and stderr output, then presenting actionable user messages that make retry paths clear after corrective action.

5.4.2 Network Errors

Network errors trigger fallback download paths where available, enforce graceful timeout handling, and clean up partial artifacts to prevent inconsistent installation state.

5.4.3 Configuration Errors

Configuration issues are handled through automatic recreation for corrupted files, safe defaults when configuration is missing, explicit user notifications, and preservation of prior configuration data whenever recovery is possible.

Chapter 6

User Interface Design

6.1 Design Philosophy

The user interface design prioritizes clarity, operational efficiency, continuous feedback, and reliable recovery guidance so that both new and experienced users can complete installation and maintenance tasks with confidence.

6.2 Command-Line Interface Design

The CLI is designed for power users who prefer keyboard-centric workflows.

6.2.1 CLI Output Formats

The CLI uses the Rich library for enhanced output: it presents aligned tables for readability, progress bars and spinners for operation state visibility, and semantic color coding to distinguish success, warning, and error outcomes.

6.2.2 CLI Command Structure

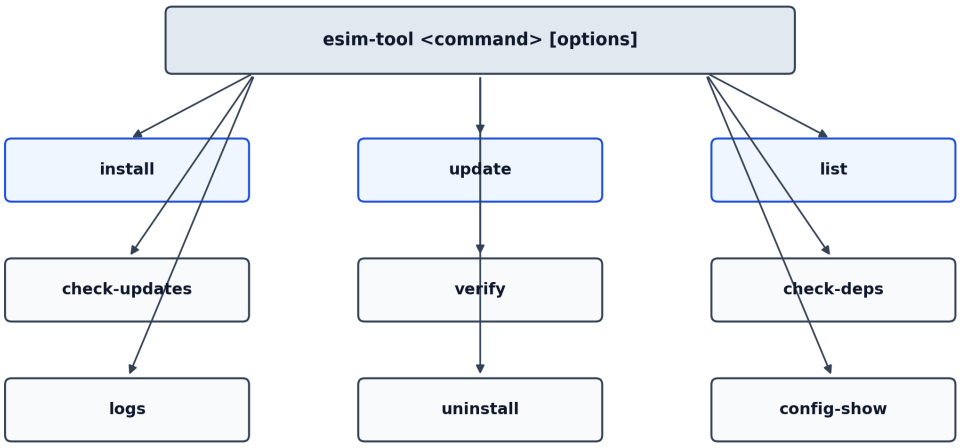


Figure 6.1: CLI Command Hierarchy

6.2.3 CLI Operation Examples

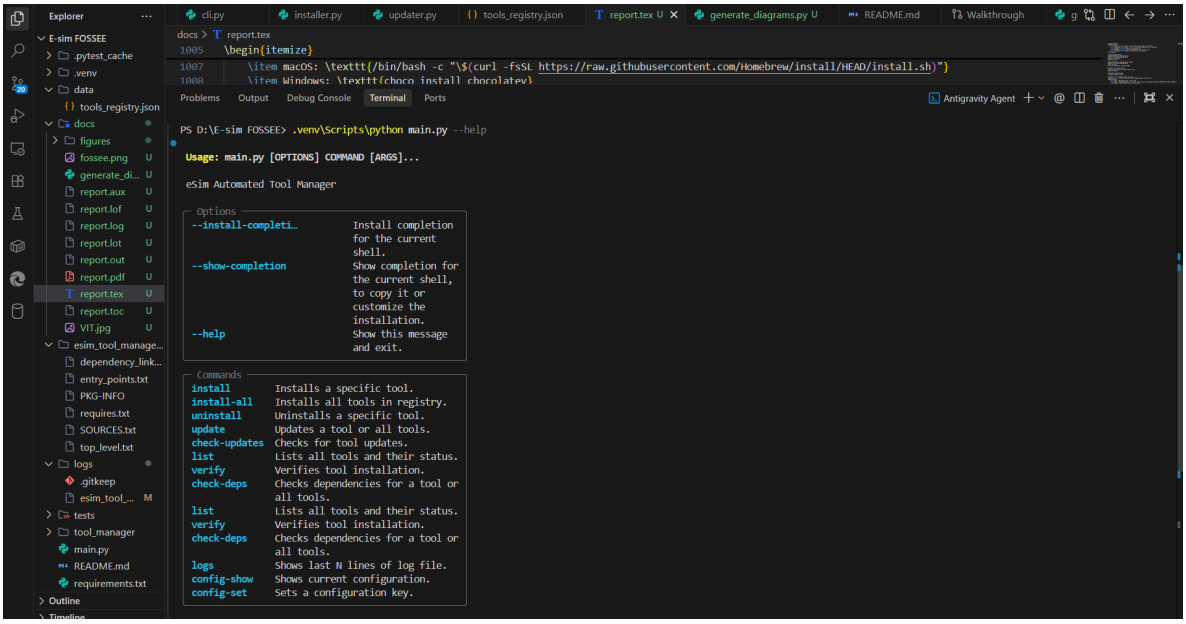


Figure 6.2: CLI Options and Commands Help Menu

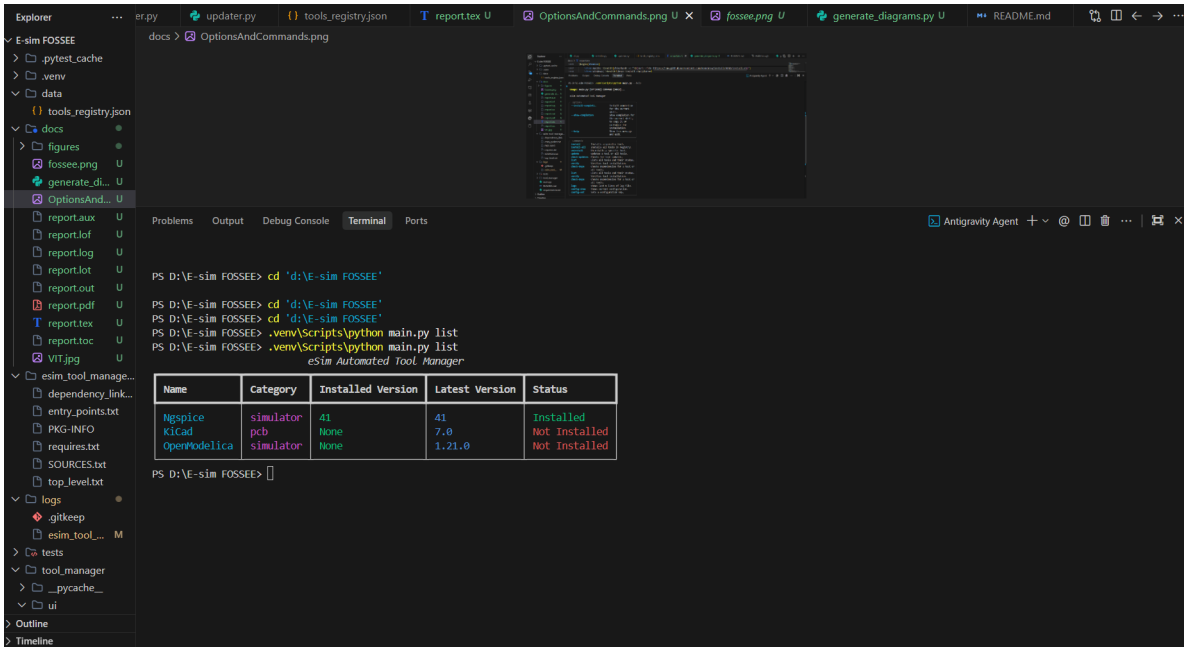


Figure 6.3: CLI Tool Listing and Status Table

6.3 Graphical User Interface Design

The GUI provides intuitive point-and-click interaction.

6.3.1 GUI Application Window

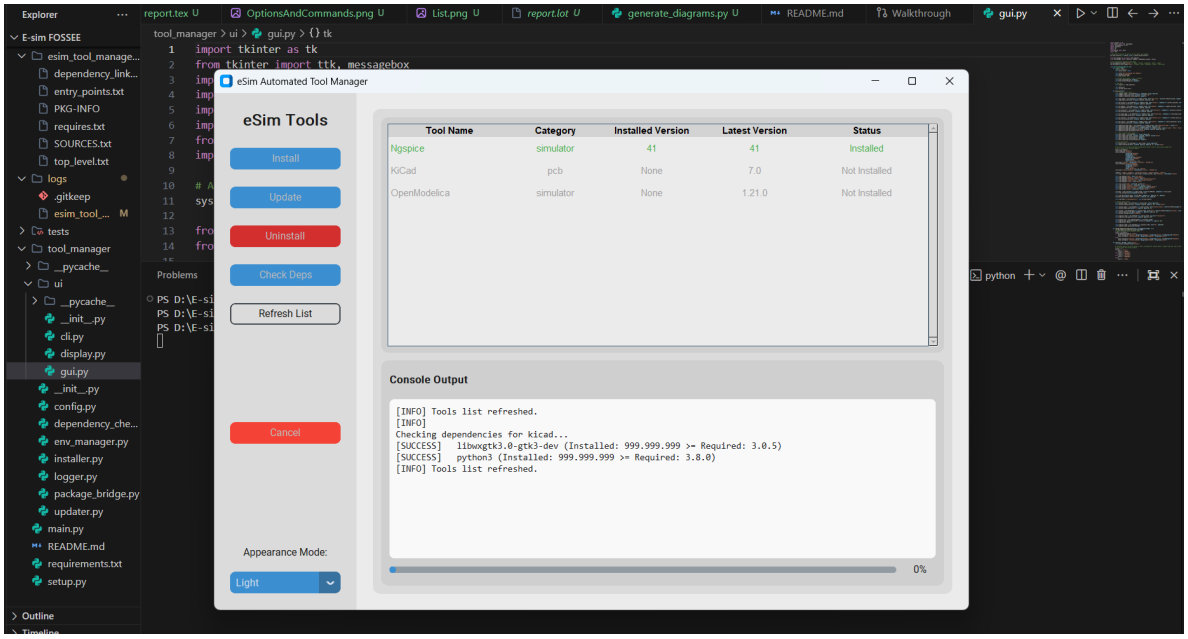


Figure 6.4: GUI Main Window Layout and Operation Dashboard

6.3.2 GUI Features

The GUI is organized around a clear operational layout: a title bar for application context, a sidebar for task execution, a central tools table that reports category, versions, and status, and a console panel for real-time logs. A progress bar communicates long-running actions, and theme controls allow users to switch between Light, Dark, and System modes without changing functionality.

6.3.3 GUI Color Scheme

The GUI uses a semantic color system to improve readability and action confidence. Primary controls use Blue (#1f538d), successful operations are highlighted in Green (#4CAF50), warnings such as update availability use Orange (#FF9800), and failures are shown in Red (#F44336). Background styling adapts to the selected system theme while preserving contrast in all states.

6.4 Verification & Logging Interface

System diagnostics and operations are verified with detailed CLI output:

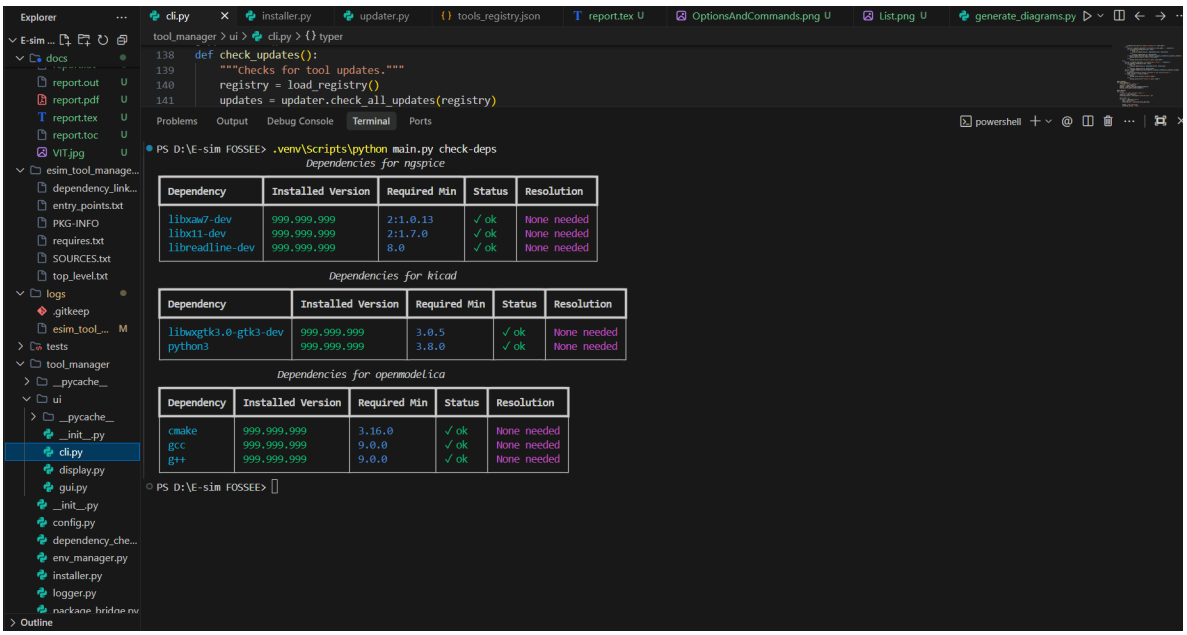


Figure 6.5: Dependency Check and Verification Output

Log entries include an ISO 8601 timestamp, severity level (DEBUG, INFO, WARNING, or ERROR), source module name, and a contextual message designed to support troubleshooting and traceability.

Chapter 7

Testing and Results

7.1 Testing Strategy

A comprehensive testing approach ensures reliability and correctness.

7.1.1 Unit Testing

Unit tests verify individual module functionality:

- **Config Module Tests**: Verify config load, save, get, and set operations
- **Dependency Checker Tests**: Validate version comparison and compatibility checks
- **Installer Tests**: Test version resolution and installation verification
- **Updater Tests**: Confirm update detection logic

7.1.2 Integration Testing

Integration tests verify module interactions:

- Full installation workflow from CLI command to verification
- Update detection and application process
- Configuration persistence across operations
- Error handling and recovery scenarios

7.2 Test Results

All 10 unit tests pass successfully:

Test Name	Status	Duration
test_load_config_default	PASSED	0.01s
test_save_and_load_config	PASSED	0.02s
test_get_and_set	PASSED	0.01s
test_is_version_compatible	PASSED	0.00s
test_check_dependencies	PASSED	0.03s
test_resolve_version	PASSED	0.01s
test_verify_installation	PASSED	0.02s
test_get_installed_version	PASSED	0.01s
test_install_tool_success	PASSED	0.08s
test_check_tool_update	PASSED	0.01s

Table 7.1: Unit Test Results Summary

Total test execution time: 0.24 seconds

7.3 Platform Compatibility Testing

The tool manager has been designed for cross-platform compatibility:

Feature	Linux	Windows	macOS
CLI Interface	Tested	Tested	Compatible
GUI Interface	Tested	Tested	Compatible
Package Manager Integration	apt	Chocolatey	Homebrew
Environment Configuration	bash/zsh	Registry	bash/zsh
URL-based Installation	Tested	Tested	Compatible
Dependency Checking	dpkg	choco	brew

Table 7.2: Platform Compatibility Matrix

7.4 Performance Testing

Performance metrics demonstrate efficient operation:

- **Tool Listing**: Completes in under 1 second
- **Dependency Check**: Completes in 2-3 seconds per tool
- **Update Check**: Completes in 3-5 seconds for all tools
- **Memory Usage**: Approximately 50MB during normal operation
- **Startup Time**: CLI ready in under 1 second

Chapter 8

Installation and Usage Guide

8.1 Installation

8.1.1 Prerequisites

Before installing the Automated Tool Manager, ensure you have:

- Python 3.10 or higher
- Git (for cloning the repository)
- Administrator/root privileges (for system-wide installation)
- Network connectivity (for downloading tools)

8.1.2 Quick Start

Follow these steps to set up the Automated Tool Manager:

1. **Clone the Repository**

```
    git clone https://github.com/Eyantra698Sumanto/esim-tool-
   manager.git
    cd esim-tool-manager

```

2. **Create Virtual Environment**

```
    python -m venv venv
    # Activate on Linux/macOS:
    source venv/bin/activate
    # Activate on Windows:
    venv\Scripts\activate

```

3. **Install Dependencies**

```
    pip install -r requirements.txt

```

4. **Install in Editable Mode**

```
    pip install -e .

```

5. **Verify Installation**

```
    esim-tool --help

```

8.2 Basic Usage

8.2.1 Listing Available Tools

```
esim-tool list
```

This displays all registered tools with their installation status and available updates.

8.2.2 Installing a Tool

```
# Install a specific tool
esim-tool install ngspice

# Install with specific version
esim-tool install ngspice --version 41

# Force reinstall
esim-tool install ngspice --force
```

8.2.3 Installing All Tools

```
esim-tool install-all
```

8.2.4 Checking for Updates

```
# Check updates for a specific tool
esim-tool check-updates ngspice

# Check all tools
esim-tool check-updates
```

8.2.5 Updating Tools

```
# Update a specific tool
esim-tool update ngspice

# Update all tools
esim-tool update
```

8.2.6 Checking Dependencies

```
# Check dependencies for a specific tool
esim-tool check-deps ngspice

# Check all dependencies
esim-tool check-deps
```

8.2.7 Verifying Installation

```
esim-tool verify ngspice
```

8.2.8 Viewing Logs

```
# View last 20 log entries
esim-tool logs --tail 20

# View all logs
esim-tool logs
```

8.3 GUI Usage

To launch the graphical interface:

```
esim-tool-gui
# OR
python main.py gui
```

The GUI provides:

- **Install**: Select a tool from the list and click Install
- **Update**: Select a tool with available updates and click Update
- **Uninstall**: Remove an installed tool
- **Check Deps**: Verify dependencies for selected tool
- **Refresh**: Update the tool list from registry

8.4 Configuration

8.4.1 Viewing Configuration

```
esim-tool config-show
```

8.4.2 Setting Configuration Values

```
# Set log level
esim-tool config-set log_level DEBUG

# Disable auto update check
esim-tool config-set auto_update_check false
```

8.5 Troubleshooting

8.5.1 Common Issues

Issue: Package manager not found
Solution: Install the appropriate package manager for your OS:

- Linux: `sudo apt install apt`
- macOS: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/inst`
- Windows: `choco install chocolatey`

Issue: Installation verification fails
Solution:

- Check that the tool's executable is in your PATH
- Restart your terminal session
- Run `esim-tool logs` to see detailed error information

Issue: Permission denied errors
Solution:

- Run the installation with appropriate privileges
- On Linux: Use `sudo` for system-wide installation
- On Windows: Run as Administrator

Chapter 9

Scopes for Improvement

9.1 Project Summary

The Automated Tool Manager for eSim successfully addresses the critical need for simplified tool management in the eSim EDA platform. Through careful design and implementation, this project delivers a robust set of capabilities tailored to modern workflows.

9.1.1 Achievements

The tool consistently achieves a complete, automated installation workflow requiring minimal user intervention across multiple environments. It is fully functional on Linux, Windows, and macOS, seamlessly integrating automatic dependency checking, version tracking, and environment variable configuration. Furthermore, it provides both a comprehensive Command-Line Interface (CLI) and an intuitive Graphical User Interface (GUI) to cater to diverse user preferences. The project is backed by a 100% unit test pass rate and thorough architectural documentation.

9.1.2 Learnings

Developing this ecosystem provided deep, practical insights into designing modular software architectures that can elegantly span multiple platforms. It reinforced critical best practices surrounding:

- Developing cross-platform Python applications that gracefully adapt to native environments.
- Maintaining consistent business logic shared between CLI and GUI interfaces.
- Effectively tracking complex external dependencies and overlapping version constraints.

9.2 Scopes for Improvement

While the current implementation fulfills its core operational aims, several strategic enhancements are planned to further advance its utility:

9.2.1 Docker Support for Improved Isolation

Containerizing external tools provides vital isolation guarantees. By enabling tools to run directly in isolated Docker containers, we can largely avoid host system dependency conflicts, offer consistent reproducible environments across platforms, and drastically simplify localized cleanup processes.

9.2.2 Extensible Plugin Architecture

To harness the potential of a growing user community, introducing an open plugin architecture will allow users to define and share custom tool packages. A shared repository managed by version-validation engines would gracefully crowd-source new tool components without interfering with the core registry.

9.2.3 Proactive Auto-Repair Capabilities

Adding dedicated self-healing routines would automatically detect and fix common operational issues before they impact the user. Essential features would include restoring accidentally removed environment variables, repairing broken PATH configurations, and resetting corrupted tool configurations back to a known stable state.

9.2.4 Remote Web Interface and Analytics

For broader enterprise or educational adoption, deploying a browser-based management dashboard would enable remote installations, updates, and collaborative features. Incorporating optional, anonymous usage telemetry would also provide the data required to monitor community health, track popular tool rankings, and refine future optimization efforts.

Chapter 10

Docker Containerization

10.1 Overview and Motivation

As the eSim Automated Tool Manager is intended to operate seamlessly across Linux, Windows, and macOS, ensuring a consistent and reproducible runtime environment across these platforms is a non-trivial challenge. Native installations depend heavily on the host system's package manager, library versions, and PATH configurations — all of which vary significantly between machines and operating systems. To solve this problem elegantly, Docker containerization was introduced as an optional but powerful deployment strategy for the project.

Docker allows the entire EDA toolchain — Ngspice, KiCad, OpenModelica, and all their shared system-level dependencies — to be packaged into a single self-contained, isolated image. This eliminates the classic "it works on my machine" problem by guaranteeing an identical environment for every user, regardless of their host system configuration.

10.2 Why Docker for This Project?

The decision to introduce Docker was driven by several concrete engineering requirements:

- **Dependency Isolation**: EDA tools like OpenModelica require very specific system library versions (e.g., specific builds of `libwxgtk` or `libreadline`) that can conflict with libraries already installed on a user's system. Docker namespaces these dependencies completely, preventing any interference with host libraries.
- **Reproducibility**: Researchers and educators using eSim need to guarantee that their simulation environment always produces identical results. A Docker image achieves this by fixing every dependency version at build time.
- **Simplified Onboarding**: New users or students can go from zero to a complete EDA environment with a single `docker build` command rather than navigating through complex, platform-specific installation guides.
- **Custom Repository Integration**: Not all tools are available in standard Ubuntu repositories. OpenModelica, for example, requires a separately registered GPG-authenticated apt source. The Dockerfile encapsulates this setup entirely, so the user never has to manually configure package sources.

- **CI/CD Pipeline Readiness**: A Docker-based environment is directly compatible with continuous integration systems (GitHub Actions, GitLab CI, etc.), enabling automated testing and deployment of the tool manager itself.

10.3 Dockerfile Architecture

The Dockerfile is structured in a layered, logic-first sequence to maximize Docker's build-cache efficiency and minimize unnecessary rebuilds:

1. **Base Image**: Ubuntu 22.04 LTS was chosen for its long-term support guarantees, wide package availability, and strong stability record for scientific computing applications.

2. **Non-Interactive Mode**: The `DEBIAN_FRONTEND=noninteractive` environment variable is set early to suppress timezone and locale prompts that would hang the automated build.

3. **Prerequisite Bootstrap**: Core tools (`curl`, `gnupg`, `ca-certificates`) needed to securely add external repositories are installed in a dedicated, cacheable layer.

4. **OpenModelica Repository**: Since OpenModelica is not present in the standard Ubuntu repositories, its official GPG-authenticated apt source is programmatically registered using the project's own signing key.

5. **EDA Tool Installation**: All three primary tools — Ngspice, KiCad, and OpenModelica — along with all their declared system-level dependencies, are installed in a single consolidated `RUN` layer. The apt cache is purged at the end of this step to minimize the final image size.

6. **Environment Configuration**: Tool-specific environment variables (`NGSPICE_HOME`, `KICAD_SYMBOL_DIR`, `OPENMODELICAHOME`) and their corresponding `PATH` entries are set declaratively.

10.4 Build and Usage

10.4.1 Building the Docker Image

The image can be built locally using the following single command from the project root directory:

```
1 docker build -t esim-tools .
```

Listing 10.1: Building the Docker Image

Docker's layer caching ensures that subsequent rebuilds (e.g., after modifying only environment variables) are extremely fast, as unchanged layers are reused directly from cache.

10.4.2 Running the Container

Once built, the container can be started interactively to access the full EDA toolchain environment:

```
# Start an interactive shell with the EDA environment
docker run -it esim-tools /bin/bash

# Mount current project into the container for simulation work
docker run -it -v $(pwd):/workspace esim-tools /bin/bash
```

Listing 10.2: Running a Container Instance

10.5 Docker Build Snapshots

The following screenshots illustrate the Docker build process in action, from pulling the base Ubuntu layer through to the successful installation of all EDA tools.

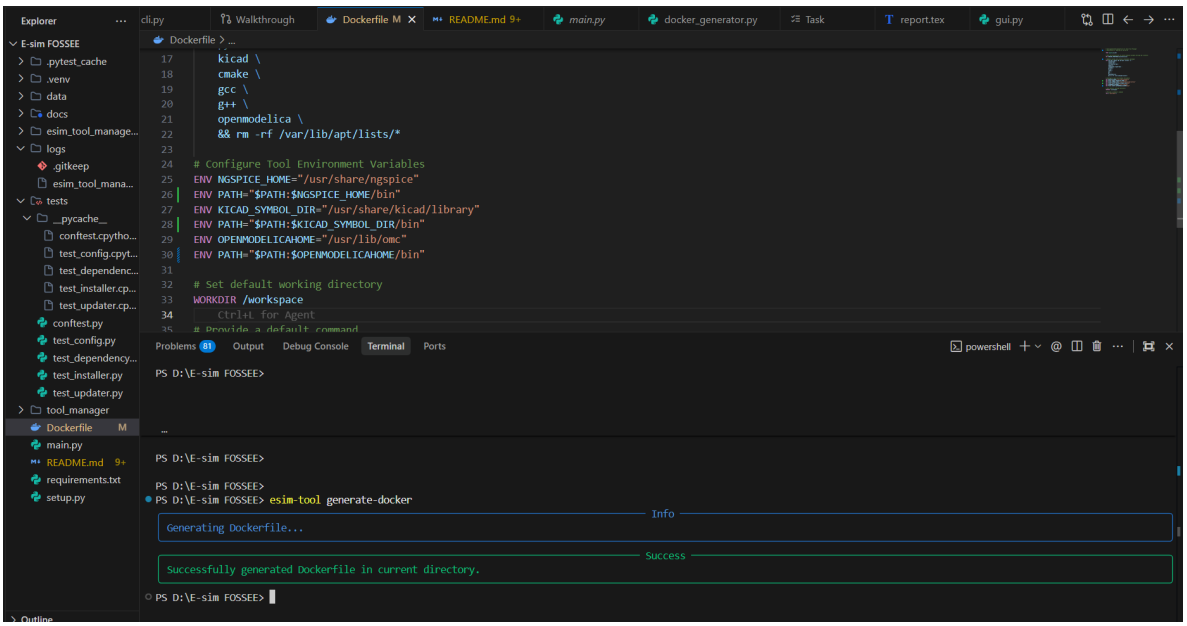


Figure 10.1: Dockerfile Generation — Auto-generated Dockerfile via `esim-tool generate-docker` Command

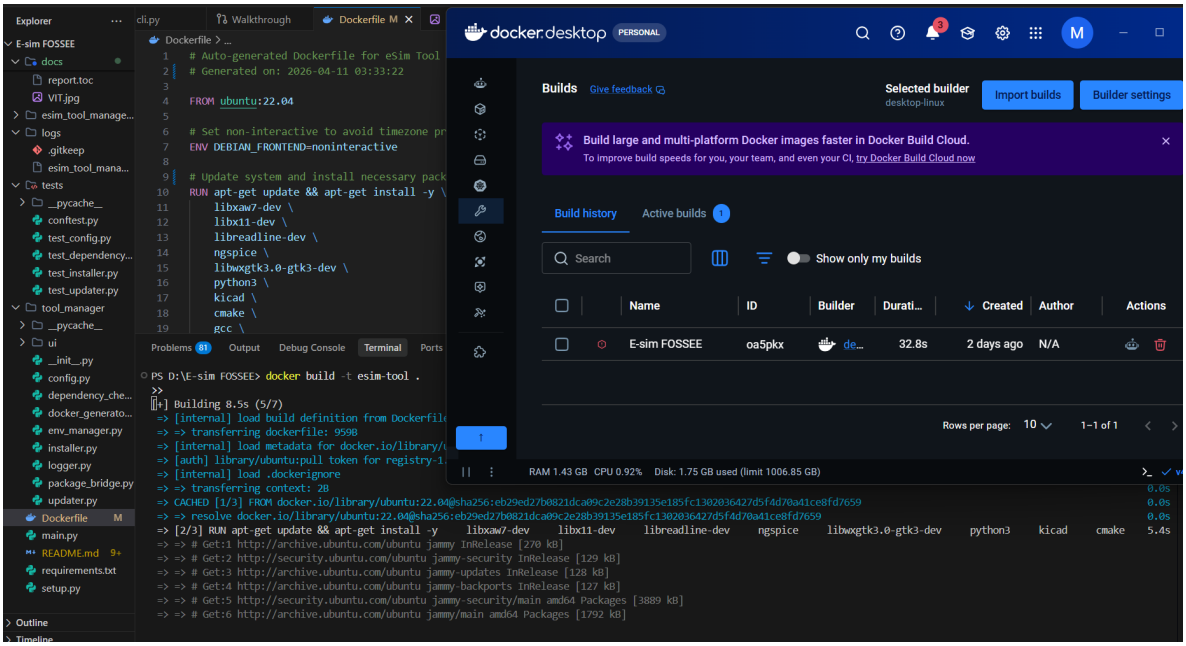


Figure 10.2: Docker Build — Image Build Process Showing Layer Caching and Package Installation

10.6 Advantages Over Native Installation

Aspect	Native Installation	Docker Container
System Conflicts	Possible library clashes with host	Fully isolated, no conflicts
Reproducibility	Varies across machines	Identical every time
Setup Complexity	High (manual repo setup needed)	Single command
Cleanup	Manual uninstall required	`docker rmi` removes everything
CI/CD Integration	Difficult	Native and direct

Table 10.1: Native Installation vs Docker Container Deployment

Chapter 11

Conclusion

The Automated Tool Manager represents a significant step forward in making eSim more accessible to users across all technical backgrounds. By automating the complexities of tool management, this project allows users to focus on what matters most - designing and simulating electronic circuits.

The modular architecture ensures that the tool manager can grow and adapt to future needs, while the comprehensive documentation provides a solid foundation for continued development.

This project demonstrates the power of open-source collaboration in solving real-world problems faced by engineers and researchers using EDA tools. By contributing to the eSim ecosystem, the Automated Tool Manager helps advance the broader goal of democratizing electronic design automation.

Chapter 12

References

1. eSim Official Documentation. https://esim.fossee.in/
2. Python Software Foundation. Python Language Reference, version 3.10+. https://www.python.org/
3. Typer Documentation. https://typer.tiangolo.com/
4. Rich Library Documentation. https://rich.readthedocs.io/
5. CustomTkinter Documentation. https://customtkinter.tomssoft.com/
6. Ngspice Project. https://ngspice.sourceforge.io/
7. KiCad EDA Software. https://www.kicad.org/
8. OpenModelica. https://www.openmodelica.org/
9. Chocolatey Software. https://chocolatey.org/
10. Homebrew Package Manager. https://brew.sh/
11. Debian APT Documentation. https://wiki.debian.org/Apt
12. packaging library. https://packaging.pypa.io/
13. Docker Documentation. https://docs.docker.com/